THREAT MODEL AND SECURE CONFIGURATION

Security assessment report | STRIDE-based analysis

Beginner-friendly example for a small web application.

1. Objective and scope

Identify realistic security threats to a small web application, assess likelihood and impact, and recommend proportionate defensive controls using STRIDE.

2. Assumptions and scope

• Because the screenshot does not name a particular application, this report uses a representative small web application with user registration/login, a browser-based interface, an application/API server, a database, an administrator dashboard, logging, and optional external email/payment services.

• The application is assumed to be internet-accessible and to store basic account/profile information. No real credentials, production data, or live systems are used in this example.

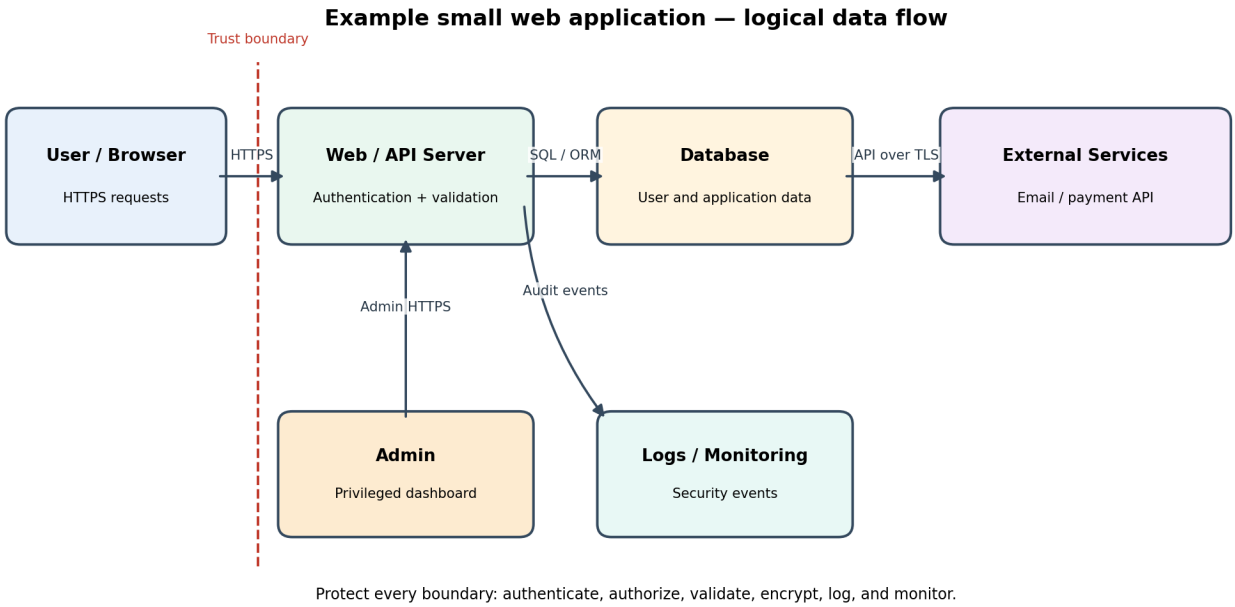
• All testing and configuration checks should be performed only in an environment you own or are explicitly authorised to assess.

In scope: browser traffic, authentication, application/API logic, database, admin functions, logs and optional third-party services. This is not a production penetration test.

3. System assets

ID	Asset	Security needs
A1	User account data	Confidentiality, integrity
A2	Authentication credentials and sessions	Confidentiality, integrity
A3	Application source code and configuration/secrets	Confidentiality, integrity
A4	Database records	Confidentiality, integrity, availability
A5	Admin functions and privileges	Integrity, availability
A6	Audit logs and security events	Integrity, availability

4. Threat model diagram and data flows



Main flows: browser to web/API over HTTPS; server to database through parameterised queries/ORM; server to external providers over TLS; administrator to protected admin interface; application to protected logs. Trust boundaries include the public internet edge, application/database boundary, third-party integrations, and ordinary-user/admin privilege boundary. Validate all data crossing boundaries.

5. STRIDE threat analysis and risk register

Scoring is qualitative (Low / Medium / High / Very High), not a measured probability.

ID / STRIDE	Threat scenario	Weakness / cause	L	I	Priority	Controls
T1 / Spoofing	Attacker uses stolen credentials or a hijacked session to impersonate a user/admin.	Weak passwords, credential reuse, missing MFA, insecure session handling.	High	High	Critical	MFA for admins; strong password hashing; secure cookies; session rotation and expiry; login rate limiting.
T2 / Tampering	Attacker changes another user's data or alters requests/records.	Missing server-side authorisation and input validation; insecure direct object references.	Medium	High	High	Enforce object-level authorisation on every request; validate inputs server-side; use parameterised queries; integrity checks where appropriate.
T3 / Repudiation	A user or attacker denies performing a sensitive action and logs cannot establish what happened.	Insufficient or modifiable audit logging; missing timestamps and actor identifiers.	Medium	Medium	Medium	Record authentication and sensitive actions; use UTC timestamps, actor, action, target and outcome; restrict log access; centralise and protect logs.
T4 / Information Disclosure	Private data, credentials, tokens, or error details are exposed.	SQL injection, excessive API responses, public storage, verbose errors, missing TLS.	Medium	High	High	TLS everywhere; least-privilege database accounts; output minimisation; safe error messages; secret manager; access-controlled backups.
T5 / Denial of Service	Application becomes unavailable due to request floods or resource exhaustion.	No rate limits, expensive endpoints, unbounded uploads, weak capacity controls.	Medium	High	High	Rate-limit login and sensitive endpoints; request/body size limits; timeouts; pagination; monitoring; tested backup and recovery plan.
T6 / Elevation of Privilege	Ordinary user gains admin access or performs admin-only actions.	Broken role checks, default admin credentials, insecure admin endpoints.	Low	Very High	High	Deny by default; server-side role checks; separate admin accounts; MFA; remove default accounts; review permissions and test access boundaries.

6. Prioritised hardening plan

Priority	Threats	Actions
P0 — Before release	T1, T2, T4, T6	Enforce authentication and server-side authorisation; protect admin accounts with MFA; parameterise database queries; enable HTTPS; remove default credentials and debug mode.
P1 — High priority	T5, T3	Add rate limits and resource caps; protect audit logs; monitor authentication failures and sensitive actions; verify backup restoration.
P2 — Continuous improvement	All threats	Patch dependencies; review security headers and CORS; conduct periodic access-control tests; update the threat model when features or data flows change.

7. Secure configuration checklist

• **Transport:** Redirect HTTP to HTTPS; enable TLS 1.2+ (prefer TLS 1.3); enable HSTS after confirming HTTPS works throughout.

• **Authentication:** Hash passwords with Argon2id or bcrypt using a suitable work factor; never store plaintext passwords; require MFA for administrators; rate-limit login and password-reset attempts.

• **Sessions:** Use Secure, HttpOnly and SameSite cookies; rotate session IDs after login/privilege changes; expire inactive sessions; invalidate sessions on logout/password reset.

• **Authorisation:** Check permissions on the server for every request and every object; use deny-by-default; separate user and admin roles; prevent direct-object-reference access.

• **Input and database:** Validate inputs on the server using allow-lists; encode output; use parameterised queries/ORM safely; limit upload types and sizes; avoid dynamic SQL.

• **Configuration and secrets:** Remove default accounts and sample pages; disable debug mode; remove unused services; keep secrets out of source control; use environment-specific secret storage; apply least privilege.

• **Browser security:** Set Content-Security-Policy, X-Content-Type-Options, Referrer-Policy and an appropriate frame-ancestors policy; configure CORS with explicit trusted origins.

• **Logging and monitoring:** Log login failures, privilege changes, access denials and sensitive data changes; do not log passwords, full tokens or unnecessary personal data; alert on unusual activity.

• **Dependencies:** Inventory dependencies; update supported versions; scan for known vulnerabilities; pin and review dependencies; remove unused packages.

• **Backups and recovery:** Encrypt backups; restrict access; define retention; test restoration; document incident contacts and recovery steps.

• **Verification:** Run authorised dependency/static checks and web security tests in a staging environment; retest after fixes; keep evidence and dates.

8. Verification / evidence checklist

ID	Test	Safe verification	Expected result
TC1	Unauthenticated access	Request a protected page/API without a session.	Request is rejected; no private data is returned.
TC2	Role separation	Attempt an admin action using a normal user account.	Server returns 403/denies action; event is logged.
TC3	Object-level access	User A requests User B's record by changing an object identifier.	Access is denied; no record data is disclosed.
TC4	Input handling	Submit unexpected characters and oversized values in a staging form.	Input is safely rejected/handled; no database error or code execution.

ID	Test	Safe verification	Expected result
TC5	Session protection	Inspect session cookie flags and test logout/session expiry.	Secure, HttpOnly, SameSite are set; expired/logged-out session cannot be reused.
TC6	Rate limiting	Make repeated login attempts in an authorised test environment.	Requests are throttled; monitoring records the event.

Evidence: dated screenshots or redacted test output, configuration notes, dependency scan summary, and fixes/retests. Never include real passwords, tokens or personal data.

9. Conclusion

The leading risks are account/session compromise, broken authorisation, information disclosure and privilege escalation. Prioritise authentication, server-side access control, HTTPS, safe database access and removal of insecure defaults. Add rate limiting, audit logging, monitoring, dependency maintenance and tested backups. Update this model when the real application architecture changes.

10. References

• OWASP Threat Modeling — https://owasp.org/www-community/Threat_Modeling

• OWASP Application Security Verification Standard — https://owasp.org/www-project-application-security-verification-standard/

• OWASP Cheat Sheet Series — https://cheatsheetseries.owasp.org/

• Microsoft STRIDE threat modeling — https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats

Submission note: This is an illustrative model because the screenshot does not identify a specific application. Replace the assumptions with your actual app details before submitting.